MODULE 21 · DOMAIN 2 - CONTAINER ORCHESTRATION

Troubleshooting

A method, and the usual causes of Pending, crash loops, pull failures, OOM kills, NotReady nodes and DNS trouble.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 21 of 35

admin@sthithapragna.com

Work **from the symptom inwards**: read the status, read the events, then follow the layer they point to.

admin@sthithapragna.com

A method that works every time

01
get
What state is it in? STATUS, READY, RESTARTS.

02
describe
Why? The Events at the bottom.

03
logs
With --previous: its last words before crashing.

04
Check the layer
Node, network, storage, config, quota.

05
Fix and verify
Change one thing, then watch it recover.

ON THE EXAM

get, describe, logs: in that order. Most exam troubleshooting answers come from one of the three.

From symptom to likely cause

SYMPTOM	USUALLY MEANS	LOOK AT
Pending	Cannot be scheduled, or a volume cannot bind	describe Pod: FailedScheduling
ContainerCreating, for long	Image still pulling, volume mount, or network plugin	describe Pod events
ErrImagePull / ImagePullBackOff	Wrong name or tag, or no registry credentials	describe Pod events
CrashLoopBackOff	The app starts, then exits or is killed, repeatedly	logs --previous
Running, but 0/1 READY	The readiness probe is failing	describe Pod: probe events
Deployment with no Pods	Pods rejected: quota or Pod Security	describe ReplicaSet

ON THE EXAM

The status tells you which layer. That alone rules out most answer options.

Reading a Pending Pod's events

EVENT SAYS...	CAUSE	MODULE
Insufficient cpu / memory	Requests do not fit any node	12
had untolerated taint	Every node has a taint the Pod does not tolerate	12
didn't match node affinity/selector	No node has the required labels	12
unbound immediate PersistentVolumeClaims	The claim cannot be satisfied	20
volume node affinity conflict	The disk is in another zone	20
No events at all	The scheduler itself may be down	3

ON THE EXAM

The FailedScheduling message lists every reason, node by node. Read it before changing anything.

Image pull failures

- **ErrImagePull** is the first failure; **ImagePullBackOff** is the kubelet waiting before retrying.
- **A typo** in the repository, tag or registry. The commonest cause by far.
- **A private registry** without an imagePullSecret: 'pull access denied' or 'unauthorized'.
- **No image for this CPU architecture,** an unreachable registry, or a rate limit.

> ON THE EXAM
>
> The container never started, so there are no logs. The answer is in describe's events.

Exit codes worth knowing

EXIT CODE	MEANS	TYPICAL CAUSE
0	Exited successfully	A short task run as a Deployment
1	Application error	Bad config, missing variable, a crash
126 / 127	Cannot execute / command not found	Wrong command or entrypoint
137	Killed with SIGKILL (128 + 9)	OOMKilled, or killed after the grace period
143	Stopped with SIGTERM (128 + 15)	A normal graceful shutdown

ON THE EXAM

137 plus reason OOMKilled means the memory limit. 137 alone means something killed it.

CrashLoopBackOff: the usual causes

- **Missing configuration:** a ConfigMap, Secret or variable the app expects at start-up.
- **A dependency is down:** the database is unreachable and the app exits.
- **Wrong command or arguments** in the Pod spec.
- **A liveness probe kills a slow starter** before it is ready. A startup probe fixes that.
- **OOMKilled,** over and over, under load.

> ON THE EXAM
>
> Always read logs with --previous: the current container may have only just started.

A troubleshooting session

```
$ kubectl get pods -n quayside
NAME                     READY   STATUS             RESTARTS   AGE
booking-api-5b8c4d-q8r2m   0/1     CrashLoopBackOff   6          9m
$ kubectl logs booking-api-5b8c4d-q8r2m -n quayside --previous
Error: environment variable DB_PASSWORD is not set
$ kubectl describe pod booking-api-5b8c4d-q8r2m -n quayside
    Last State:  Terminated
      Reason:    Error
      Exit Code: 1
```

3 What state: crashing, six restarts.

5 Why: the previous container's last words.

9 Exit code 1: an application error, not OOM.

THE TAKEAWAY

Fix: add the Secret reference to the Pod template, then roll out.

OOMKilled versus Evicted

OOMKILLED

One container, its own limit

- The kernel kills the container
- It exceeded its own memory limit
- Restarts inside the same Pod
- Exit code 137; RESTARTS climbs

EVICTED

Whole Pod, node pressure

- The kubelet removes the whole Pod
- The node itself ran short of memory or disk
- Pod ends Failed; a controller replaces it
- Often another Pod's fault

THE TIEBREAKER

OOMKilled: raise the limit or fix the app. Evicted: look at the node and at other Pods' requests.

Node conditions

CONDITION	TRUE MEANS
Ready	The kubelet is healthy and can run Pods. False or Unknown is the problem
MemoryPressure	The node is low on memory; evictions may follow
DiskPressure	The node is low on disk space
PIDPressure	Too many processes on the node
NetworkUnavailable	The node's network is not configured

ON THE EXAM

kubectl describe node shows every condition with a reason. Start there for NotReady.

Why a node goes NotReady

- **The kubelet stopped:** crashed, or not started after a reboot.
- **The container runtime is down,** so the kubelet cannot run anything.
- **No working network plugin:** the classic fresh-cluster cause.
- **Resource exhaustion, an expired certificate,** or the node losing its network.
- **On the node itself,** check the kubelet's service status and its logs.

> ON THE EXAM
>
> Remember module 5: after about five minutes NotReady, Pods on the node are evicted.

A Service is unreachable: check in this order

01
DNS
Does the name resolve from a Pod?

›

02
Service
Does it exist, with the right port and targetPort?

›

03
Endpoints
Does its EndpointSlice list ready Pods?

›

04
Pod
Is the app listening on targetPort?

›

05
Policy
Is a NetworkPolicy blocking the path?

ON THE EXAM

An empty EndpointSlice is the single most common cause: a selector mismatch or no ready Pods.

DNS problems

- **Are the CoreDNS Pods running** in kube-system, and is the kube-dns Service there?
- **Test from inside a Pod:** nslookup kubernetes.default from a small utility Pod.
- **An egress policy blocking port 53** breaks every lookup, as module 19 showed.
- **A short name from another namespace** fails; use service.namespace.

> ON THE EXAM
>
> If kubernetes.default resolves but your Service does not, the problem is your name or namespace, not DNS.

Eight failures. For each, ask: what does the status say, and which layer does it point to?

admin@sthithapragna.com

> The booking API's new Pods show ImagePullBackOff. The image is registry.quayside.example/booking-api:2.4.1, and the tag exists.
>
> describe shows: pull access denied, repository may require authorization.

What is the fix?

A Raise the container's memory limit

B Restart the kubelet on every node

C Add an imagePullSecret for the registry

D Change the Pod's restartPolicy to Never

◆ PAUSE AND DECIDE

What is the fix?

A Raise the container's memory limit
Memory has nothing to do with pulling images.

B Restart the kubelet on every node
The kubelet is working; the registry refused it.

C Add an imagePullSecret for the registry
The registry is private; nodes need credentials.

D Change the Pod's restartPolicy to Never
Restart policy applies after a container starts.

THE SIGNAL

'Pull access denied' from a private registry: missing credentials.

> A new Pod stays Pending. Its event reads: 0/5 nodes are available: 5 node(s) had untolerated taint {dedicated: ml}.

What is the cause?

A. Every node has a taint the Pod does not tolerate

B. The nodes lack enough memory for its requests

C. The image cannot be pulled from the registry

D. The scheduler has stopped working

◆ PAUSE AND DECIDE

What is the cause?

A Every node has a taint the Pod does not tolerate

Exactly what the event says, on all five nodes.

B The nodes lack enough memory for its requests

That would read Insufficient memory.

C The image cannot be pulled from the registry

Image pulls happen after scheduling, not before.

D The scheduler has stopped working

A dead scheduler writes no events at all.

THE SIGNAL

Read the FailedScheduling reason literally. It names the cause.

> The payment worker restarts every few minutes during the evening rush. describe shows Last State: Terminated, Reason: OOMKilled, Exit Code: 137.
>
> Its memory limit is 256Mi.

What is the most appropriate fix?

A Raise the container's CPU limit

B Add a readiness probe to the payment worker

C Set imagePullPolicy to Always

D Raise the memory limit, or cut memory use

◆ PAUSE AND DECIDE

What is the most appropriate fix?

A Raise the container's CPU limit

CPU over its limit is throttled, never killed.

B Add a readiness probe to the payment worker

Readiness controls traffic, not memory.

C Set imagePullPolicy to Always

Pulling has nothing to do with memory kills.

D Raise the memory limit, or cut memory use

It exceeds its own memory limit under load.

THE SIGNAL

OOMKilled plus 137: the container's own memory limit. Raise it or fix the usage.

> A team runs a data-import script as a Deployment. The script does its work and exits successfully after five seconds.
>
> The Pod shows CrashLoopBackOff, and describe shows Last State: Terminated, Reason: Completed, Exit Code: 0.

Why is it crash-looping?

A The image is corrupt, so the container fails to start

B Deployments restart containers after success too

C The container is being OOMKilled

D A liveness probe keeps killing it

◆ PAUSE AND DECIDE

Why is it crash-looping?

A The image is corrupt, so the container fails to start

It starts fine and even completes its work.

B Deployments restart containers after success too

restartPolicy Always restarts every exit. Use a Job instead.

C The container is being OOMKilled

Exit code 0 and Completed rule out a kill.

D A liveness probe keeps killing it

A probe kill would not report Completed with code 0.

THE SIGNAL

Exit code 0 in a crash loop: finite work running as a Deployment. It should be a Job.

> After a kernel patch and reboot, node-4 shows NotReady. describe node reports the kubelet has stopped posting status.
>
> On the node, the kubelet service is inactive: it was never enabled to start at boot.

What is the right fix?

A Delete node-4 and join it again from scratch

B Start the kubelet and enable it at boot

C Drain node-4 so its Pods move elsewhere

D Restart the CoreDNS Pods in kube-system

◆ PAUSE AND DECIDE

What is the right fix?

A Delete node-4 and join it again from scratch

Drastic, and the same fault returns on the next reboot.

B Start the kubelet and enable it at boot

The kubelet is simply not running. Start it, and enable it.

C Drain node-4 so its Pods move elsewhere

Moves work away but leaves the node broken.

D Restart the CoreDNS Pods in kube-system

DNS has nothing to do with node readiness.

THE SIGNAL

NotReady with the kubelet stopped: start the kubelet, and make it start at boot.

> A booking API Pod is Running but shows 0/1 READY, and receives no traffic. Its logs look normal.
>
> The readiness probe checks port 8080 at /healthz. The app was recently changed to listen on port 9090.

What is wrong?

A The liveness probe will soon restart it

B A NetworkPolicy is blocking its traffic

C DNS cannot resolve the booking API name

D The readiness probe checks the old port

◆ PAUSE AND DECIDE

What is wrong?

A The liveness probe will soon restart it

Liveness is not mentioned, and the app is healthy.

B A NetworkPolicy is blocking its traffic

Probes come from the kubelet on the node; the probe itself fails.

C DNS cannot resolve the booking API name

DNS is irrelevant: the Pod is marked not ready.

D The readiness probe checks the old port

Probe on 8080, app on 9090: never ready, so no traffic.

THE SIGNAL

Running but not Ready, healthy logs: suspect the readiness probe itself.

> From the payment worker, nslookup booking-api.quayside fails. So do lookups of every other in-cluster name.
>
> In kube-system, the CoreDNS Pods show CrashLoopBackOff. There are no NetworkPolicies in either namespace.

Where is the problem?

A Cluster DNS itself: CoreDNS is not running

B The booking API Service's selector is wrong

C A NetworkPolicy protects the booking API

D kube-proxy is running in the wrong mode

◆ PAUSE AND DECIDE

Where is the problem?

A Cluster DNS itself: CoreDNS is not running

Every lookup fails and CoreDNS is crash-looping.

B The booking API Service's selector is wrong

A wrong selector still resolves; it just reaches nothing.

C A NetworkPolicy protects the booking API

There are no policies, and policy does not affect DNS answers.

D kube-proxy is running in the wrong mode

Lookups fail before any Service traffic is sent.

THE SIGNAL

Every name fails, not just one: cluster DNS itself.

> The export-reports Deployment shows 0/3 ready, and no Pods exist for it at all.
>
> The ReplicaSet's events say: exceeded quota: team-quota, requested: pods=1, used: pods=40, limited: pods=40.

What is the cause?

A The scheduler is down, so Pods are not placed

B The image cannot be pulled

C The namespace has reached its Pod quota

D Pod Security Admission rejects the Pods

◆ PAUSE AND DECIDE

What is the cause?

A The scheduler is down, so Pods are not placed
Pods would still exist, stuck in Pending.

B The image cannot be pulled
A pull failure needs a Pod to exist first.

C The namespace has reached its Pod quota
40 of 40 Pods used: creation is refused.

D Pod Security Admission rejects the Pods
That event would name PodSecurity, not a quota.

THE SIGNAL

Deployment with no Pods: read the ReplicaSet's events. Here, the quota.

Module 21 in five lines

- Method: **get** for state, **describe** for events, **logs --previous** for crashes.
- **Pending** is scheduling or storage; **ImagePullBackOff** is name or credentials; no logs exist.
- Exit **137 + OOMKilled** is the memory limit; **exit 0 crash loop** is finite work in a Deployment.
- **NotReady:** describe node, then kubelet, runtime, network plugin, resources.
- **Unreachable Service:** DNS, Service, EndpointSlice, Pod, policy. Empty endpoints is most common.

That completes Container Orchestration. Next domain: **Cloud Native Application Delivery**.

admin@sthithapragna.com